



CRC 2026

#3 Interfejs JDBC

Poznaj integrację baz danych z językiem Java

ING Hubs Poland Bartosz Gawron

April 2026



Agenda

- Zasady spotkania
 - Przerwa (15min w połowie zajęć ?)
- Przedstawmy się
- Bazy danych – trochę teorii
- Składnia języka Sql
- Bezpieczeństwo i kontrola dostępu w bazie danych
- Snapshots
- Wirtualna tabela
- Interfejs JDBC
- ORM (np. Hibernate)
- DDL
- Migracja danych
- Podsumowanie
 - Zaliczenie egzaminu

Przedstawmy się



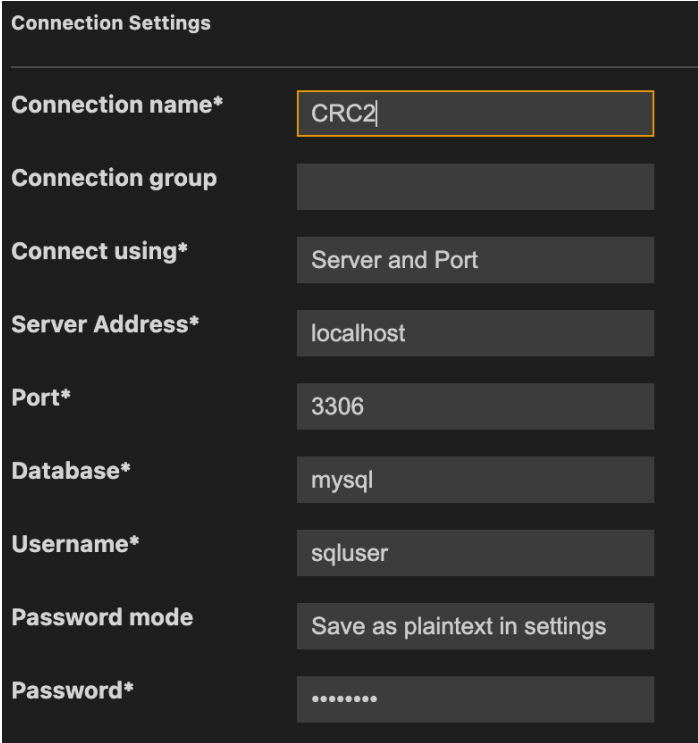
Bartosz Gawron

- bartosz.gawron@ing.com

Instalacja i konfiguracja

Instalacja i konfiguracja

- <https://www.docker.com/>
- Posiadasz Visual Code Studio
 - Zainstalowany driver do bazy **SQLTools MySQL/MariaDB & SQLTools**
 - Stworzenie obrazu
 - `docker run --name crc-2023 -p 3306:3306 -e MYSQL_ROOT_PASSWORD=Welcom0nCrc23 -d mysql`
- Polaczenie do bazy
- Inicjalizacja bazy danymi //link do pastie z sql



The image shows a 'Connection Settings' dialog box with a dark theme. It contains several input fields and dropdown menus for configuring a database connection. The fields are labeled as follows:

Field Label	Value
Connection name*	CRC2
Connection group	
Connect using*	Server and Port
Server Address*	localhost
Port*	3306
Database*	mysql
Username*	sqluser
Password mode	Save as plaintext in settings
Password*	

Instalacja i konfiguracja

Uruchom docker-compose.yml

Nawiązaj połączenie do bazy

Uruchom create_tables.sql

Loguje się do bazy `mysql -u root`

```
CREATE DATABASE MYSQLTEST;
```

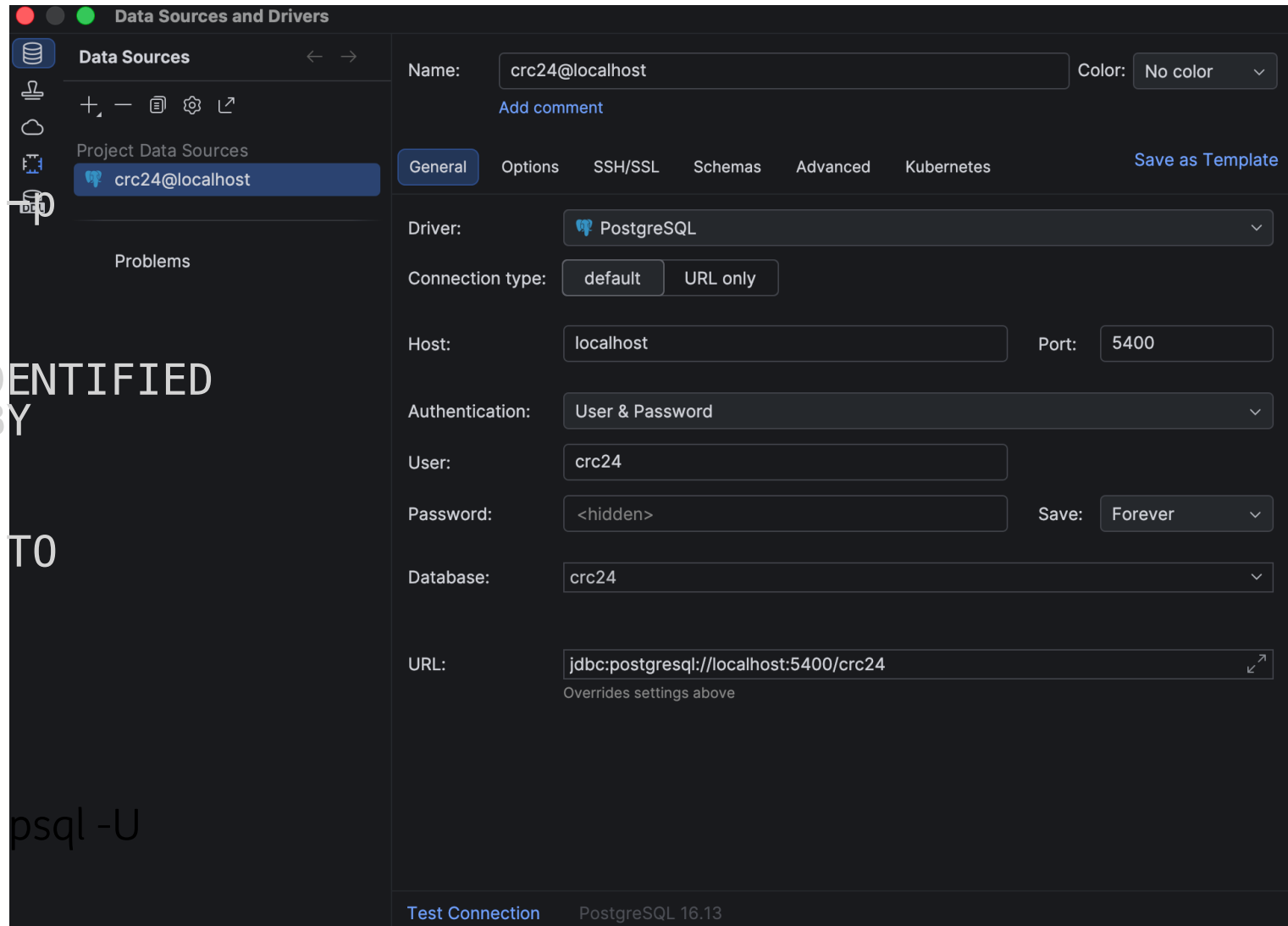
```
CREATE USER 'sqluser'@'%' IDENTIFIED  
WITH mysql_native_password BY  
'password';
```

```
GRANT ALL PRIVILEGES ON *.* TO  
'sqluser'@'%';
```

```
FLUSH PRIVILEGES;
```

---postgres

```
docker exec -it db_postgres_crc_2026 postgresql -U  
crc24
```



Wstęp do Baz danych

The image features a person in a dark blue suit, seen from the chest up, holding a glowing, three-dimensional database icon (represented by three stacked cylinders) in their right hand. The background is a light blue gradient with a network of white lines and dots, and several teal-colored squares floating around, suggesting a digital or data environment.

Definicja & ewolucja baz danych

Baza danych to zorganizowany zbiór usystematyzowanych informacji, czyli danych, zwykle przechowywany w systemie komputerowym w formie elektronicznej.

Bazę danych steruje zwykle system zarządzania bazami danych (DBMS). Dane i system DBMS oraz powiązane z nimi aplikacje razem tworzą system bazodanowy, często nazywany w skrócie bazą danych.



Typy baz danych

Relacyjne

Obiektowe

Rozproszone

Hurtownie danych

NoSQL (nierelacyjne)

Grafowe

Chmurowe

Dokumentowe

1. Relacyjne - Elementy w relacyjnej bazie danych są zorganizowane jako zbiór tabel zawierających kolumny i wiersze. Technologia relacyjnych baz danych zapewnia najbardziej efektywny i elastyczny sposób na uzyskanie dostępu do uporządkowanych informacji.
2. Obiektowe - Informacje w baize mają postać obiektów
3. Rozproszone – składa się z co najmniej dwóch plików znajdujących się w różnych lokalizacjach. Baza danych może być przechowywana na wielu komputerach znajdujących się w tej samej lokalizacji fizycznej lub rozproszonych w różnych sieciach.
4. Hurtownie –posiada Centralne repozytorium danych tzw. hurtownia danych. Przeznaczone głównie do szybkiego wykonywania zapytań i analizy.
5. NoSQL – (nierelacyjne) umożliwia przechowywanie nieusystematyzowanych i częściowo usystematyzowanych danych oraz manipulowanie nimi.
6. Grafowe – przechowuje dane w postaci encji i relacji między encjami.
7. Chmurowe – Istnieją dwa modele chmurowych baz danych: tradycyjny i baza danych jako usługa (DBaaS). W przypadku baz danych DBaaS zadania związane z administracją i konserwacją są wykonywane przez usługodawcę. O tym będzie też później opowiedziane/
8. Dokumentowe – zostały zaprojektowane pod kątem przechowywania i pobierania informacji dotyczących dokumentów oraz zarządzania nimi, to nowoczesny sposób na przechowywanie danych w formacie JSON zamiast w postaci wierszy i kolumn.

System zarządzania bazami danych (DBMS)



MySQL



Microsoft
Access



Microsoft SQL
Server



FileMaker Pro



Oracle
Database



dBASE

DBMS (Data Base Managment System)

System stanowiący interfejs pomiędzy bazą danych, a jego użytkownikami końcowymi lub programami.

Obszary zastosowań baz danych

- Systemy rezerwacji biletów
- Obsługa transakcji gotówkowych w banku
- Księgowość
- itp.

Bazy danych mają bardzo szerokie zastosowanie w wielu dziedzinach.

- Podajcie przykłady

Tradycyjne zastosowania baz danych obejmują takie dziedziny jak:

- Obsługa wpłat i wypłat w banku.
- Rezerwacja biletów lotniczych.
- Przeglądanie katalogu bibliotek UW w internecie.
- Zakupy w supermarkecie.
- Kartoteki pracowników (kadry i płace).
- Księgowość.

Wraz z rozwojem informatyki pojawiło się coraz więcej rozwiązań, takich jak:

- Multimedialne bazy danych: obrazy, muzyka, filmy.
- Geograficzne systemy informacyjne
- Systemy analizy danych
- Poszukiwanie prawidłowości w danych (Data mining).
- Naukowe i medyczne bazy danych
- Wyszukiwarki informacji: Google, Bing itp.

Użytkownicy w serwisach internetowych

Wyzwania związane z bazami danych

Znaczny wzrost ilości danych

Zapewnienie bezpieczeństwa danych

Zaspokojenie popytu

Zarządzanie bazami, infrastrukturą i utrzymanie

Eliminowanie ograniczeń skalowalności

Zapewnienie zgodności z wymaganiami przechowywania danych

ACID

Atomicity

operacja lub zestaw operacji objętych **transakcją** zostanie **wykonana w całości** lub **anulowana**.

Consistency

“baza jest w dobrym stanie”

Isolation

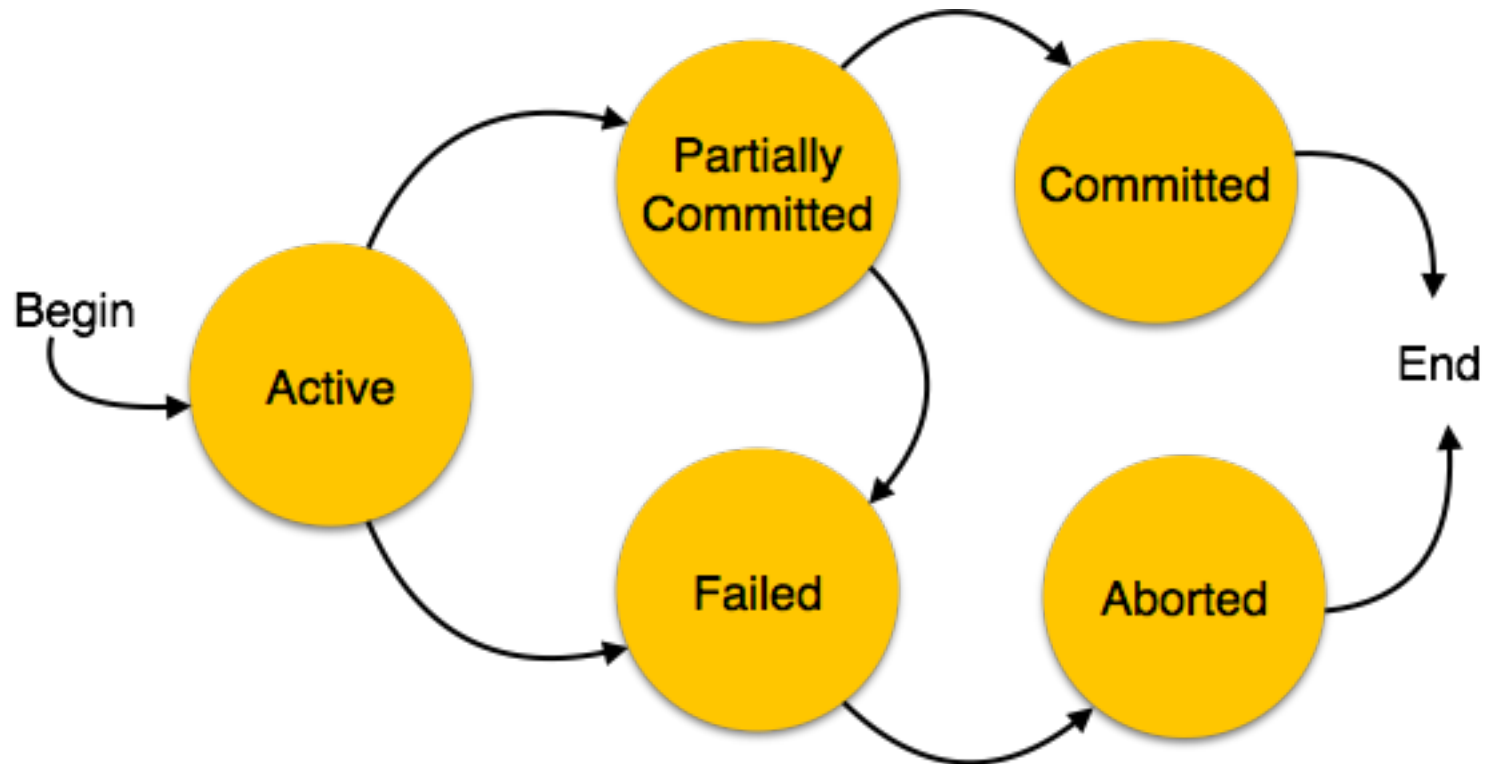
jednocześnie wykonywane **transakcje** są od siebie odizolowane

Durability

zapisane dane nie zostaną zapomniane (bez względu na sytuacje awaryjne - awaria sprzętu / bazy).

Transakcja

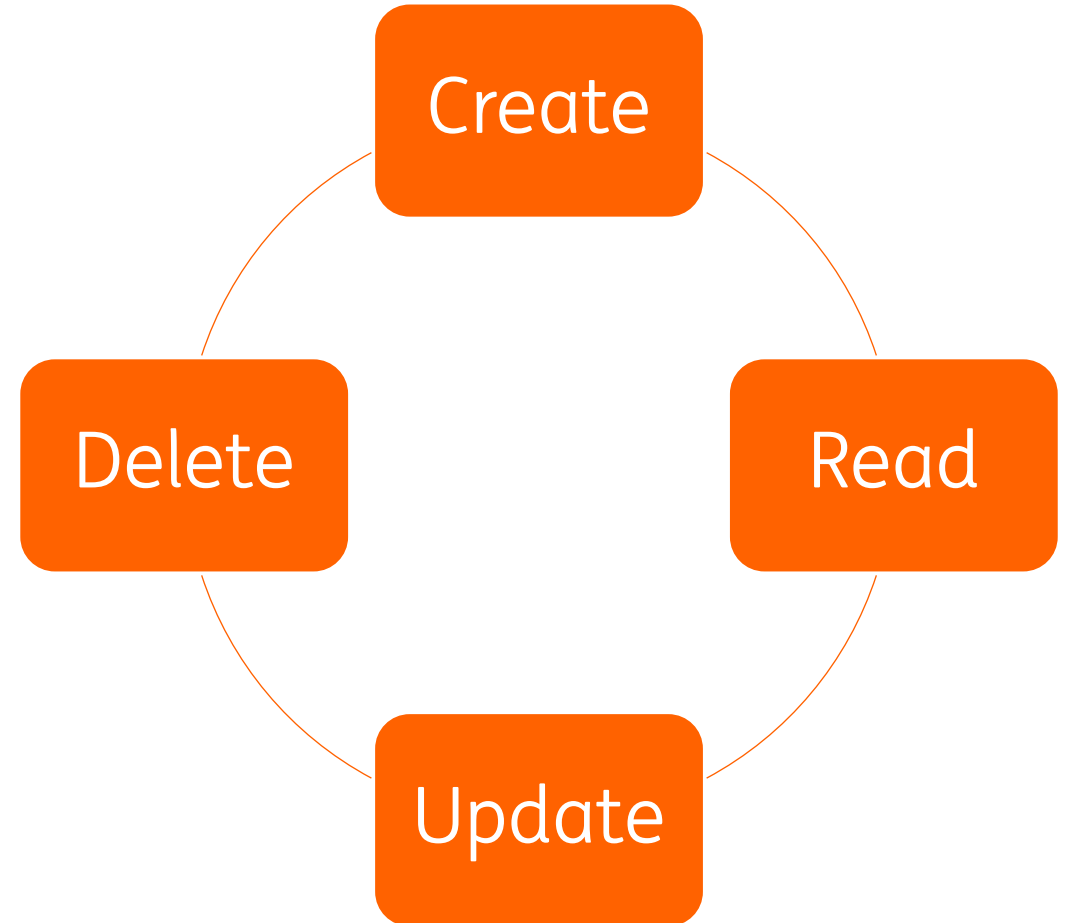
Jest serią jednej lub więcej operacji wykonywanych jako pojedyncza, atomowa jednostka.



CRUD

Zalety ze stosowania CRUD-a

- Standaryzacja
- Uproszczony proces rozwoju
- Łatwość utrzymania
- Doświadczenie użytkownika
- Modularność i elastyczność
- Skalowalność
- Kompatybilność z różnymi technologiami



SQL

(Structured Query Language) - język programowania używany do tworzenia zapytań dotyczących danych, ich przetwarzania i definiowania oraz zapewnienia kontroli dostępu.

Klauzula języka

- SELECT – zwraca listę pól
- FROM – zwraca listę tabel
- WHERE – określa kryteria pól
- ORDER BY – określa sposób sortowania
- GROUP BY – zwraca listę pól uporządkowaną
- HAVING – określa warunki stosowane dla pól

Terminologia języka

- Identyfikator – nazwa identyfikująca obiekt bazy danych
- Operator – słowo kluczowe reprezentujące lub modyfikujące akcję
- Stała – wartość nie ulegająca zmianie
- Wyrażenie – kombinacja identyfikatorów, operatorów, stałych i funkcji

Opracowany w latach 70 przez IBM

Agregacja danych

- AVG
- MAX/MIN
- SUM
- COUNT
- DISTINCT

Relacje

Jeden do jednego

- Każdy element występuje w każdej tabeli tylko raz. Na przykład każdy z pracowników może mieć co najwyżej jeden samochód służbowy

Jeden do wielu

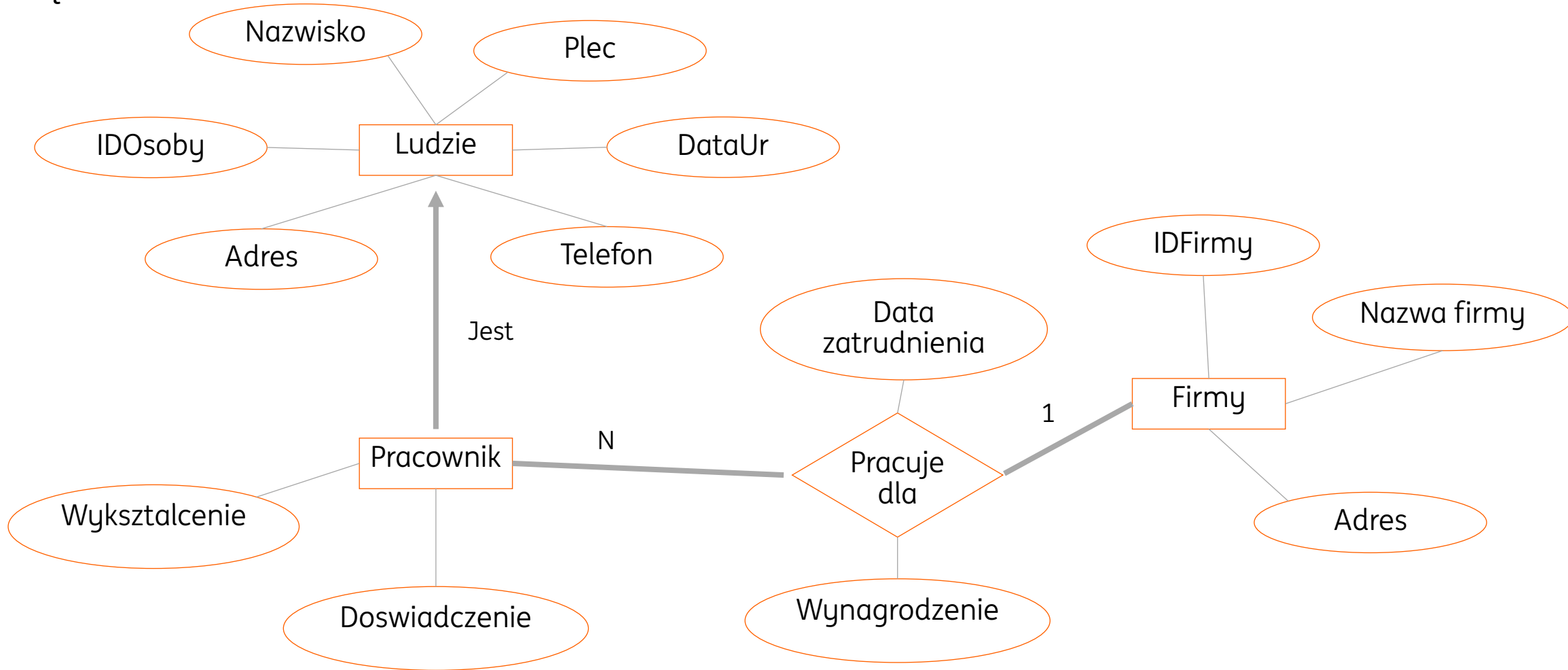
- Jeden element w jednej tabeli może być powiązany z większą liczbą elementów w innej tabeli. Na przykład jedno zamówienie zakupu może zawierać wiele produktów.

Wiele do wielu

- Element lub kilka elementów w jednej tabeli może mieć relację z jednym elementem lub kilkoma elementami w innej tabeli. Na przykład jedno zamówienie może zawierać wiele produktów, a jeden produkt może występować w wielu zamówieniach.

Przykład zadania - projektowanie tabel

Rozwiązanie



Składnia SQL

A person in a dark blue suit is shown from the chest up, holding a glowing, three-tiered database cylinder icon in their right hand. The background is a light blue gradient with a network of white lines and dots, some of which are highlighted in green and blue. The overall theme is technology and data.



Podstawy składni SQL



- INSERT
- SELECT
- UPDATE
- DELETE
- CREATE
- DROP
- ALTER
- PRIMARY KEY
- CONSTRAINT
- PROCEDURE
- VIEWS
- JOIN
- GROUP BY
- ORDER BY, WHERE, HAVING
- UNION

CREATE

- `CREATE TABLE table_name (
 column1 datatype,
 column2 datatype,
 column3 datatype,

);`
- `CREATE TABLE Persons (
 PersonID int,
 LastName varchar(255),
 FirstName varchar(255),
 Address varchar(255),
 City varchar(255)
);`

INSERT

- `INSERT INTO table_name (column1, column2, column3, ...) VALUES (value1, value2, value3, ...);`
- `INSERT INTO table_name VALUES (value1, value2, value3, ...);`
- `INSERT INTO Customers (CustomerName, ContactName, Address, City, PostalCode, Country) VALUES ('Cardinal', 'Tom B. Erichsen', 'Skagen 21', 'Stavanger', '4006', 'Norway');`
- Insert Data Only in Specified Columns
 - `INSERT INTO Customers (CustomerName, City, Country) VALUES ('Cardinal', 'Stavanger', 'Norway');`

SELECT

- `SELECT * FROM table_name;`
- `SELECT CustomerName, City FROM Customers;`

UPDATE

- `UPDATE table_name`
 `SET column1 = value1, column2 = value2, ...`
 `WHERE condition;`
- `UPDATE Customers`
 `SET ContactName = 'Alfred Schmidt', City= 'Frankfurt'`
 `WHERE CustomerID = 1;`

DELETE

- `DELETE FROM table_name WHERE condition;`
 - `DELETE FROM Customers WHERE CustomerName='Alfreds Futterkiste';`
- `DELETE FROM table_name;`
 - `DELETE FROM Customers;`

DROP

- `DROP TABLE table_name;`
- `DROP TABLE Shippers;`
- `TRUNCATE TABLE table_name;`

ALTER

- `ALTER TABLE table_name
ADD column_name datatype;`
- `ALTER TABLE Customers
ADD Email varchar(255);`

PRIMARY KEY

```
■ CREATE TABLE Persons (  
    ID int NOT NULL PRIMARY KEY,  
    LastName varchar(255) NOT NULL,  
    FirstName varchar(255),  
    Age int  
);
```

CONSTRAINT

```
■ CREATE TABLE table_name (  
    column1 datatype constraint,  
    column2 datatype constraint,  
    column3 datatype constraint,  
    ....  
);
```

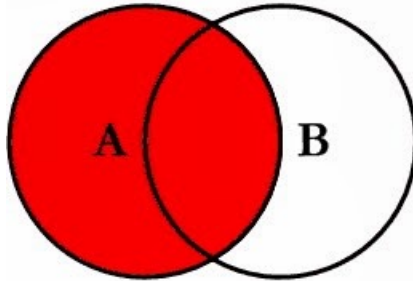
- NOT NULL - Ensures that a column cannot have a NULL value
- UNIQUE - Ensures that all values in a column are different
- PRIMARY KEY - A combination of a NOT NULL and UNIQUE. Uniquely identifies each row in a table
- FOREIGN KEY - Prevents actions that would destroy links between tables
- CHECK - Ensures that the values in a column satisfies a specific condition
- DEFAULT - Sets a default value for a column if no value is specified
- CREATE INDEX - Used to create and retrieve data from the database very quickly

PROCEDURE

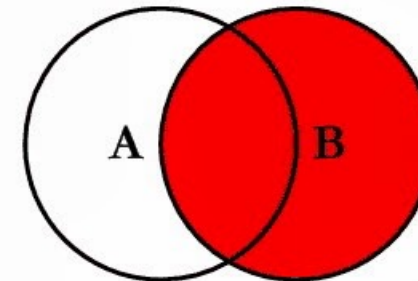
- Stworzenie procedury
 - `CREATE PROCEDURE procedure_name`
`AS`
`sql_statement`
`GO;`
 - `CREATE PROCEDURE SelectAllCustomers`
`AS`
`SELECT * FROM Customers`
`GO;`
- Wywołanie procedury
 - `EXEC procedure_name;`

JOIN

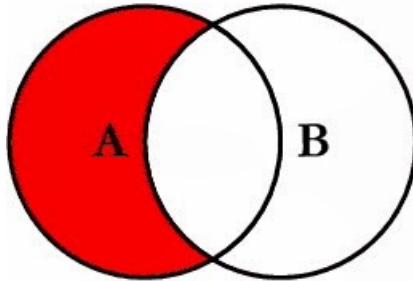
SQL JOINS



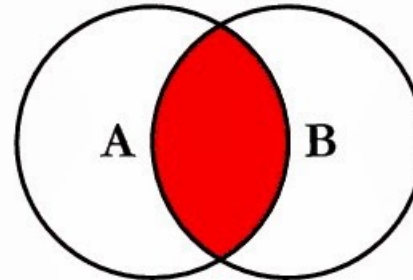
```
SELECT <select_list>  
FROM TableA A  
LEFT JOIN TableB B  
ON A.Key = B.Key
```



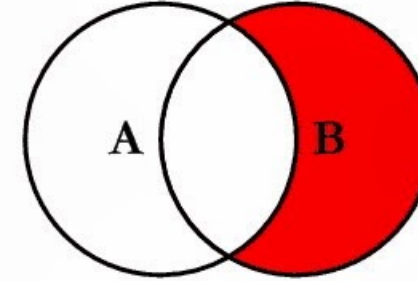
```
SELECT <select_list>  
FROM TableA A  
RIGHT JOIN TableB B  
ON A.Key = B.Key
```



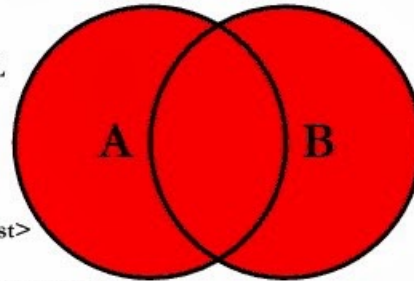
```
SELECT <select_list>  
FROM TableA A  
LEFT JOIN TableB B  
ON A.Key = B.Key  
WHERE B.Key IS NULL
```



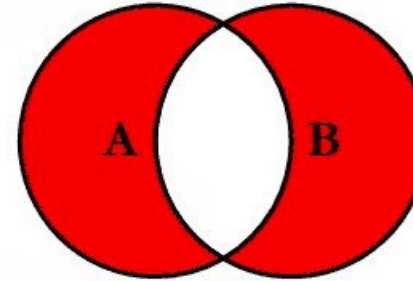
```
SELECT <select_list>  
FROM TableA A  
INNER JOIN TableB B  
ON A.Key = B.Key
```



```
SELECT <select_list>  
FROM TableA A  
RIGHT JOIN TableB B  
ON A.Key = B.Key  
WHERE A.Key IS NULL
```



```
SELECT <select_list>  
FROM TableA A  
FULL OUTER JOIN TableB B  
ON A.Key = B.Key
```



```
SELECT <select_list>  
FROM TableA A  
FULL OUTER JOIN TableB B  
ON A.Key = B.Key  
WHERE A.Key IS NULL  
OR B.Key IS NULL
```

GROUP BY

- `SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
ORDER BY column_name(s);`
- `SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country
ORDER BY COUNT(CustomerID) DESC;`
- `SELECT Shippers.ShipperName, COUNT(Orders.OrderID) AS NumberOfOrders FROM Orders
LEFT JOIN Shippers ON Orders.ShipperID = Shippers.ShipperID
GROUP BY ShipperName;`



COUNT()
MAX()
MIN()
SUM()
AVG()

ORDER BY

- `SELECT column1, column2, ...
FROM table_name
ORDER BY column1, column2, ... ASC|DESC;`
- `SELECT * FROM Customers
ORDER BY Country DESC;`
- `SELECT * FROM Customers
ORDER BY Country ASC, CustomerName DESC;`

WHERE

- `SELECT column1, column2, ...
FROM table_name
WHERE condition;`
- `SELECT * FROM Customers
WHERE Country='Mexico';`

=

>

<

>=

<=

<>

BETWEEN

LIKE

IN

HAVING

- `SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
HAVING condition
ORDER BY column_name(s);`
- `SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country
HAVING COUNT(CustomerID) > 5;`
- `SELECT Employees.LastName, COUNT(Orders.OrderID) AS NumberOfOrders
FROM Orders
INNER JOIN Employees ON Orders.EmployeeID = Employees.EmployeeID
WHERE LastName = 'Davolio' OR LastName = 'Fuller'
GROUP BY LastName
HAVING COUNT(Orders.OrderID) > 25;`

UNION

- `SELECT column_name(s) FROM table1
UNION
SELECT column_name(s) FROM table2;`
- `SELECT City, Country FROM Customers
WHERE Country='Germany'
UNION ALL
SELECT City, Country FROM Suppliers
WHERE Country='Germany'
ORDER BY City;`

Summary



Bezpieczeństwo i kontrola dostępu w bazie danych

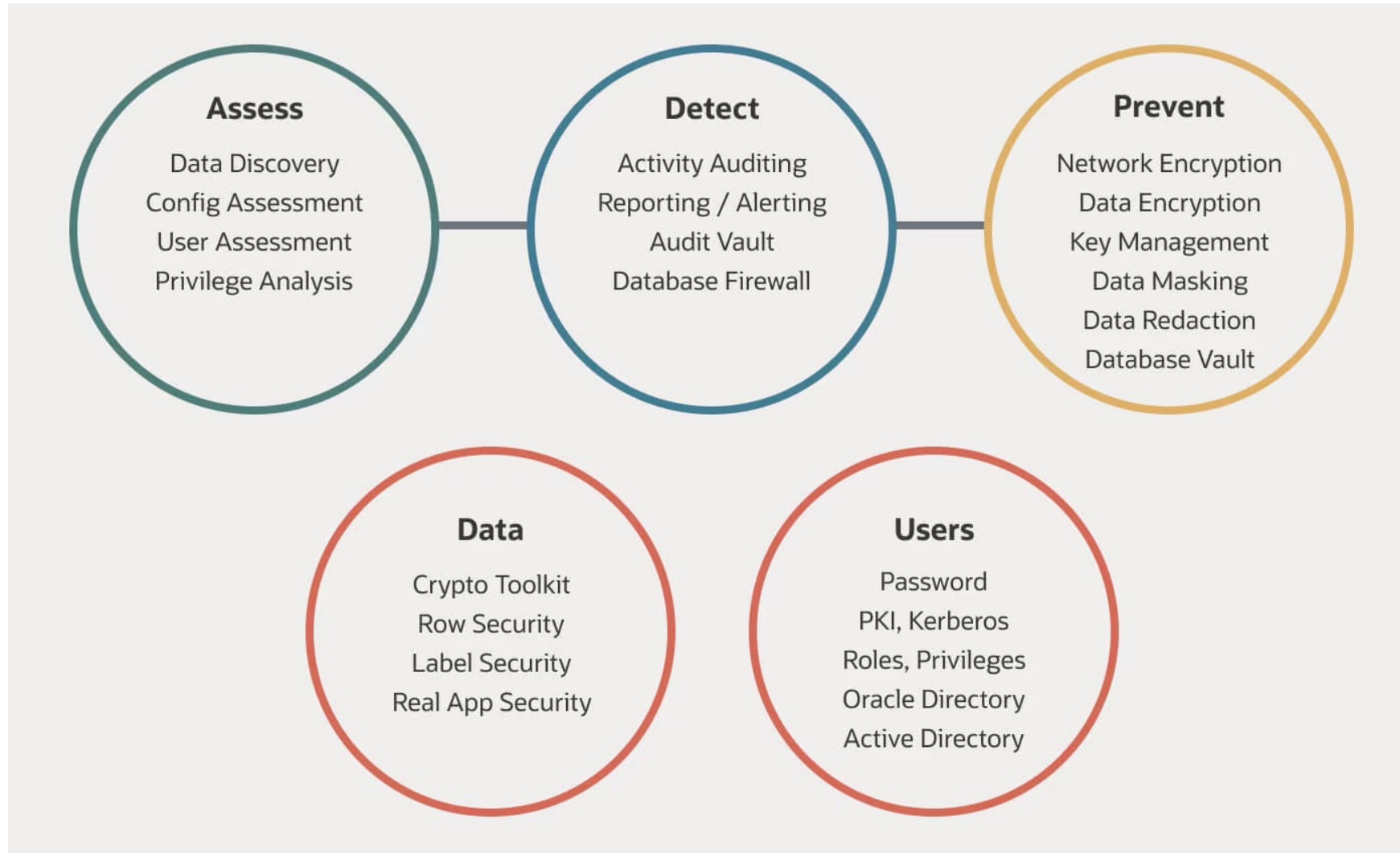


Definicja zabezpieczenia danych

Zabezpieczenie danych odnosi się do środków ochronnych stosowanych do zabezpieczania danych przed niezatwierdzonym dostępem oraz do zachowania poufności, integralności i dostępności danych.

- Najlepsze praktyki w zakresie zabezpieczenia danych obejmują m.in.
 - szyfrowanie danych,
 - zarządzanie kluczami,
 - zamazywanie danych,
 - podzbiór danych i maskowanie danych,
 - a także kontrolę dostępu uprzywilejowanych użytkowników oraz inspekcję i monitorowanie.

Najlepsze praktyki ochrony baz danych



~~CREATE USER~~ → CREATE ROLE

- CREATE USER ***name*** [[WITH] ***option*** [...]]

where ***option*** can be:

SUPERUSER | NOSUPERUSER
| CREATEDB | NOCREATEDB
| CREATEROLE | NOCREATEROLE
| INHERIT | NOINHERIT
| LOGIN | NOLOGIN
| REPLICATION | NOREPLICATION
| BYPASSRLS | NOBYPASSRLS
| CONNECTION LIMIT ***connlimit*** | [ENCRYPTED] PASSWORD '***password***' | PASSWORD NULL
| VALID UNTIL '***timestamp***' | IN ROLE ***role_name*** [, ...] | IN GROUP ***role_name*** [, ...] | ROLE
role_name [, ...] | ADMIN ***role_name*** [, ...] | USER ***role_name*** [, ...] | SYSID ***uid***

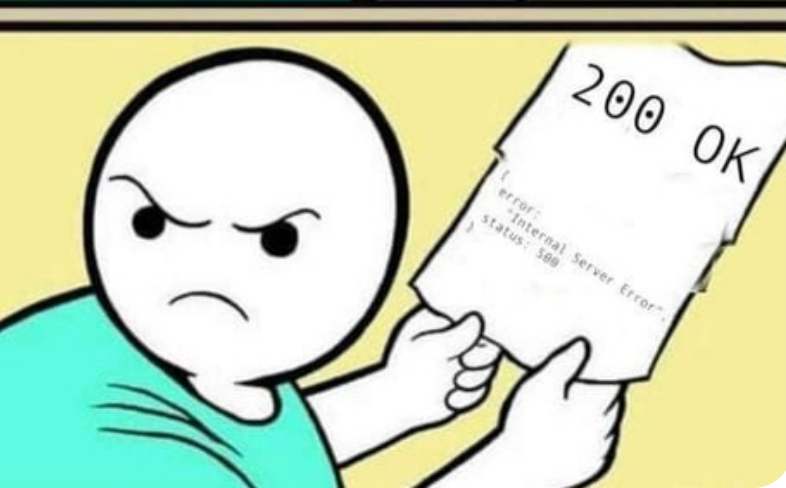
GRANT ROLES

- SELECT
- INSERT
- UPDATE
- DELETE
- TRUNCATE
- REFERENCES
- TRIGGER
- CREATE
- CONNECT
- TEMPORARY TEMP
- EXECUTE
- USAGE
- ALL PRIVILEGES

```
GRANT SELECT ON soda TO crc_viewer;  
GRANT CONNECT ON database postgres TO crc_viewer;
```


SCHEMA

- CREATE SCHEMA
- DROP SCHEMA name
- DROP SCHEMA name CASCADE;



Summary



Snaphosts





Snapshot

- **Snapshot** to odtworzenie stanu danych w określonym momencie czasu.
Pozwala „zamrozić” widok danych na potrzeby **audytu, raportów, analityki** lub **opornej wydajnościowo pracy na dużych zbiorach** — bez wpływu na bieżące transakcje.

Wirtualna tabela





View

- Upraszczają złożone zapytania – użytkownik korzysta z gotowej “tabeli logicznej”
- Izolują logikę biznesową od struktury fizycznej bazy
- Zwiększają bezpieczeństwo – można udzielić dostęp tylko dla widoku, a nie do tabeli źródłowej
- Pomagają standaryzować raporty i analizy danych.

```
1  CREATE VIEW active_users AS
2  SELECT id, name, email
3  FROM users
4  WHERE active = TRUE;
5
```

VIEWS

- `CREATE VIEW view_name AS`
`SELECT column1, column2, ...`
`FROM table_name`
`WHERE condition;`
 - `CREATE VIEW [Brazil Customers] AS`
`SELECT CustomerName, ContactName`
`FROM Customers`
`WHERE Country = 'Brazil';`
- Wywołanie widoku
 - `SELECT * FROM [Brazil Customers];`

A person in a dark suit and tie is holding a glowing, multi-layered cylinder that resembles a database or data storage unit. The cylinder has a bright white top and bottom, with glowing blue and green horizontal bands in the middle. A network of white lines and dots is overlaid on the scene, connecting various points. The background is dark and out of focus.

Materialized View

- przyspiesza zapytania działające na dużych zbiorach danych
- odciąża bazę z kosztownych operacji JOIN/AGGREGATION
- idealny w raportowaniu i analityce (BI, dashboardy, hurtownie danych)
- powtarzalne zapytania wykonują się błyskawicznie

```
1 CREATE MATERIALIZED VIEW mv_sales AS
2 SELECT user_id, SUM(total) AS total_sales
3 FROM orders
4 GROUP BY user_id;
5
```

```
1 REFRESH MATERIALIZED VIEW mv_sales;
```




View vs Materialized View



- View: gdy potrzebujesz zawsze świeżych danych
- Materialized View: gdy liczy się wydajność, a dane nie muszą być real-time

Cecha	VIEW (Widok)	MATERIALIZED VIEW (Widok zmaterializowany)
Typ	logiczny (wirtualny)	fizyczny (zapisany na dysku)
Przechowywanie danych	❌ dane nie są przechowywane – zapytanie wykonywane przy każdym odwołaniu	✅ dane są przechowywane – wynik zapytania utrwalony w bazie
Aktualność danych	✅ zawsze aktualne (pobierane bezpośrednio z tabel źródłowych)	❌ aktualne tylko na moment ostatniego REFRESH
Wydajność	⚠️ zależna od złożoności zapytania i ilości danych	✅ bardzo wysoka, szczególnie dla dużych JOIN i agregacji
Zastosowanie	uproszczenie zapytań, bezpieczeństwo, warstwa prezentacji	raporty, analityka, BI, hurtownie danych
Koszt utrzymania	✅ niski (brak odświeżania)	❌ wyższy – wymaga odświeżania (manualnie lub cyklicznie)

Summary



Interfejs JDBC

A person in a dark blue suit is shown from the chest up, holding a glowing, three-dimensional database icon (three stacked cylinders) in their right hand. The background is a light blue gradient with a network of white lines and dots, some of which are highlighted in green and blue. The overall theme is technology and data.



JDBC

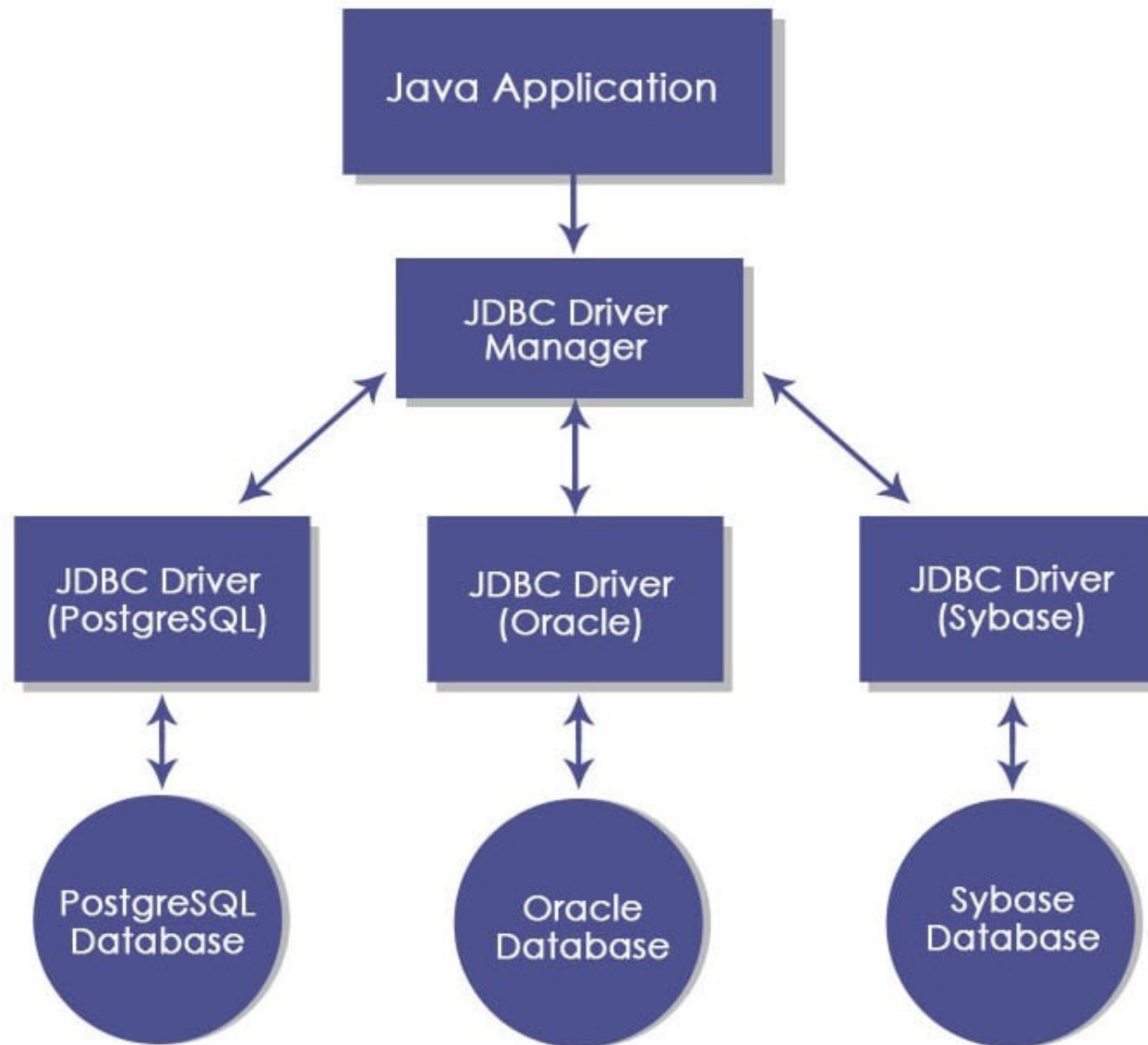
JDBC (Java Database Connectivity) to standardowy interfejs API w Javie, który umożliwia aplikacjom:

- Łączenie się relacyjną bazą danych
- Wykonywanie zapytań SQL
- Obsługę transakcji
- Pobieranie i modyfikowanie danych niezależnie od rodzaju bazy (PostgreSQL, Oracle, MySQL)

Auto-commit = true !!!

Zalety JDBC ✓	Wady JDBC ⚠
Pełna kontrola nad zapytaniami SQL	Dużo kodu „boilerplate”
Wysoka wydajność (brak warstwy pośredniej)	Ręczne mapowanie danych z ResultSet
Łatwe debugowanie zapytań SQL	Trudniejsze utrzymanie w dużych projektach
Niezależność od ORM i frameworków	Większa podatność na błędy logiczne
Bezpośrednia kontrola transakcji	Mniejsza czytelność kodu biznesowego

Schemat architektury





NamedParameterJdbcTemplate

NamedParameterJdbcTemplate to rozszerzenie JdbcTemplate, które:

pozwała używać **nazwanych parametrów SQL** (:name, :price)

zachowuje **pełną kontrolę nad SQL**

eliminuje najczęstsze problemy „? + indeksy”

SQL Injection

✗ Niebezpieczne (Statement / konkatencja)

```
1 String sql = "SELECT * FROM soda WHERE name = '" + name + "'";
2 ``
```

Jeśli name = "' OR 1=1 --":

```
1 SELECT * FROM soda WHERE name = ' ' OR 1=1 --'
```

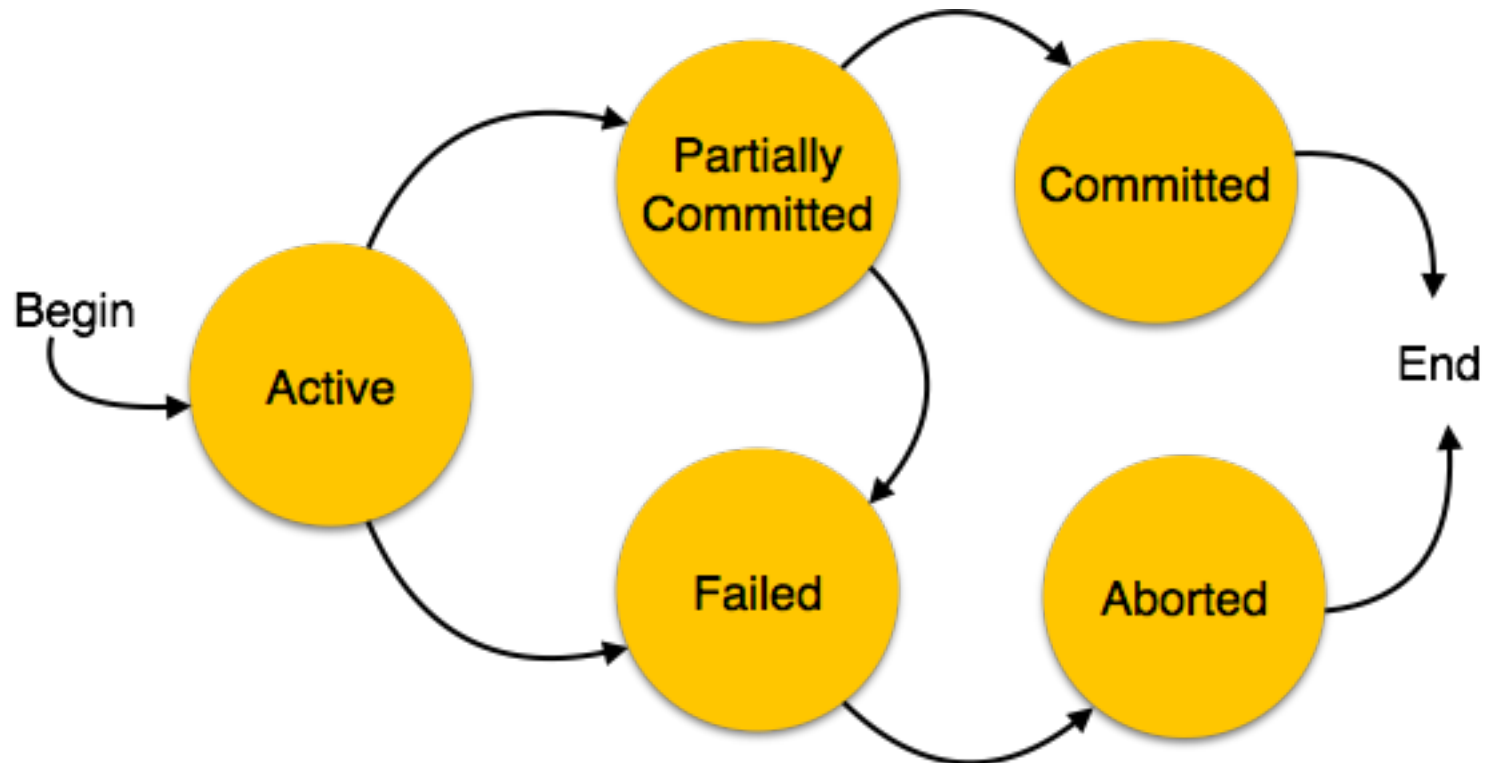
```
1 SELECT * FROM soda WHERE name = ?
2 ``
```

```
1 ps.setString(1, name);
2
```

Transakcja



Jest serią jednej lub więcej operacji wykonywanych jako pojedyncza, atomowa jednostka.



ORM - Hibernate

A person in a dark blue suit is shown from the chest up, holding a glowing, three-dimensional database icon (represented as a stack of three cylinders) in their right hand. The background is a light blue gradient with a network of white lines and dots, some of which are highlighted in green and blue. The overall theme is technology and data management.

Spring JDBC

- Cechy JDBC Api:
 - Dużo kodu przed i po zapytaniu (tworzenie połączenia, instrukcja, obsługa zestawu wyników)
 - Obsłużyć wyjątki
 - Obsłużyć transakcje
 - Stworzenie własnego mappera



ORM - Hibernate & HQL



ORM - Object-relational mapping
HQL - Hibernate Query Language

```
String hql = "FROM Employee"; Query query =  
session.createQuery(hql); List results = query.list();
```

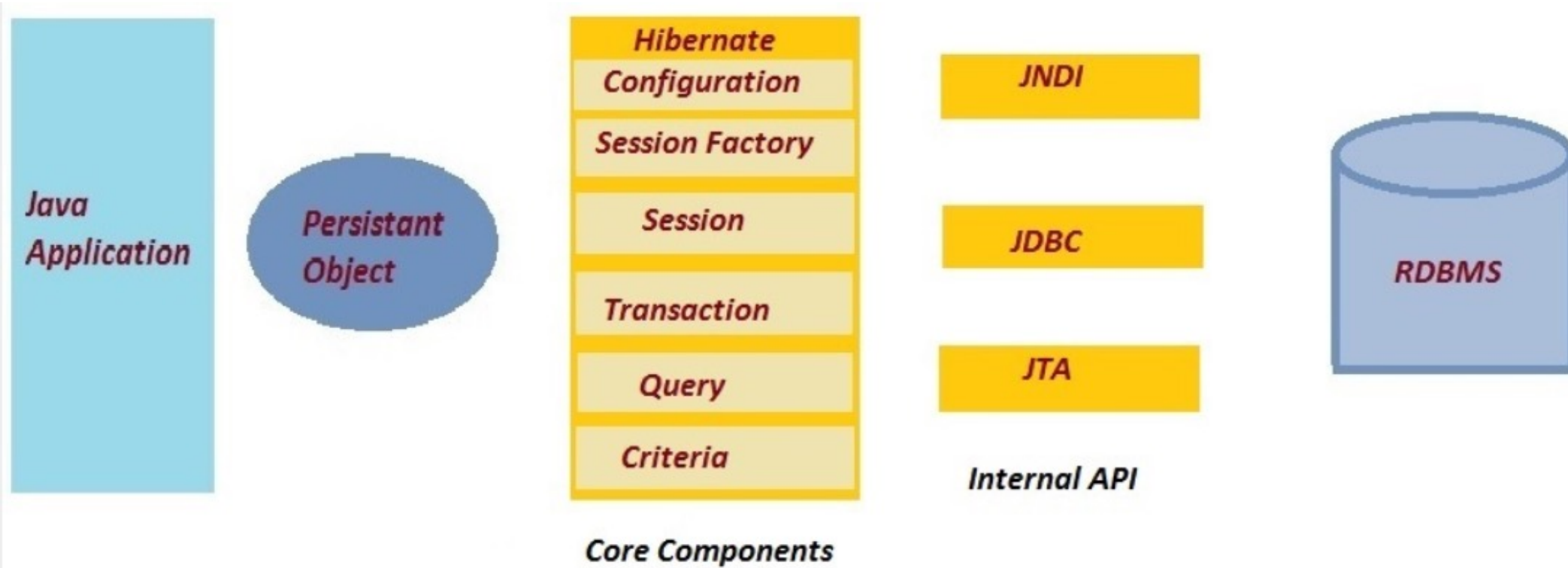
```
String hql = "UPDATE Employee set salary = :salary " + "WHERE id = :employee_id";  
Query query = session.createQuery(hql); query.setParameter("salary", 1000);  
query.setParameter("employee_id", 10); int result = query.executeUpdate();  
System.out.println("Rows affected: " + result);
```

```
String hql = "DELETE FROM Employee " + "WHERE id = :employee_id";  
Query query = session.createQuery(hql);  
query.setParameter("employee_id", 10); int result =  
query.executeUpdate(); System.out.println("Rows affected: " + result);
```





HIBERNATE



DDL <-> Entity





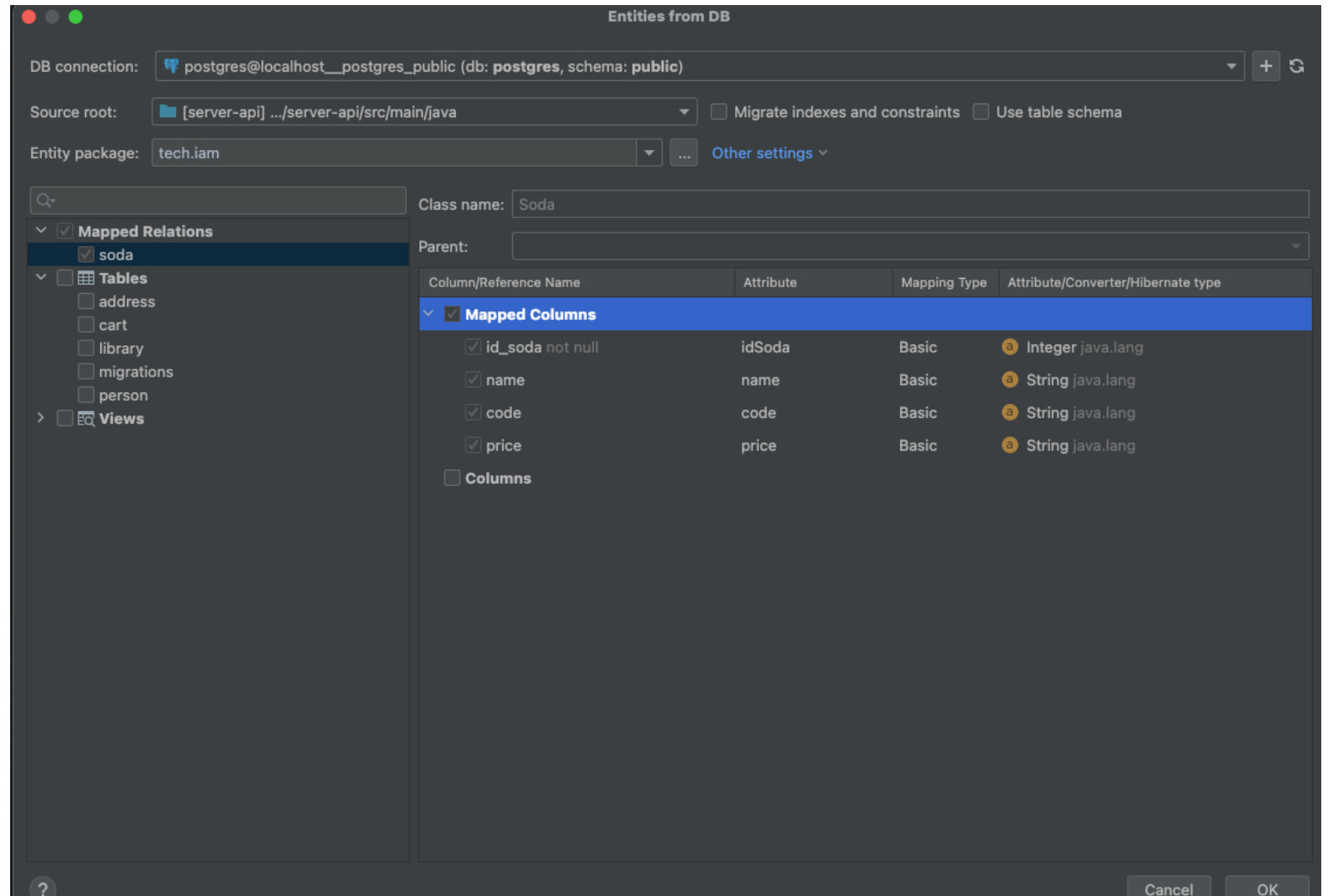
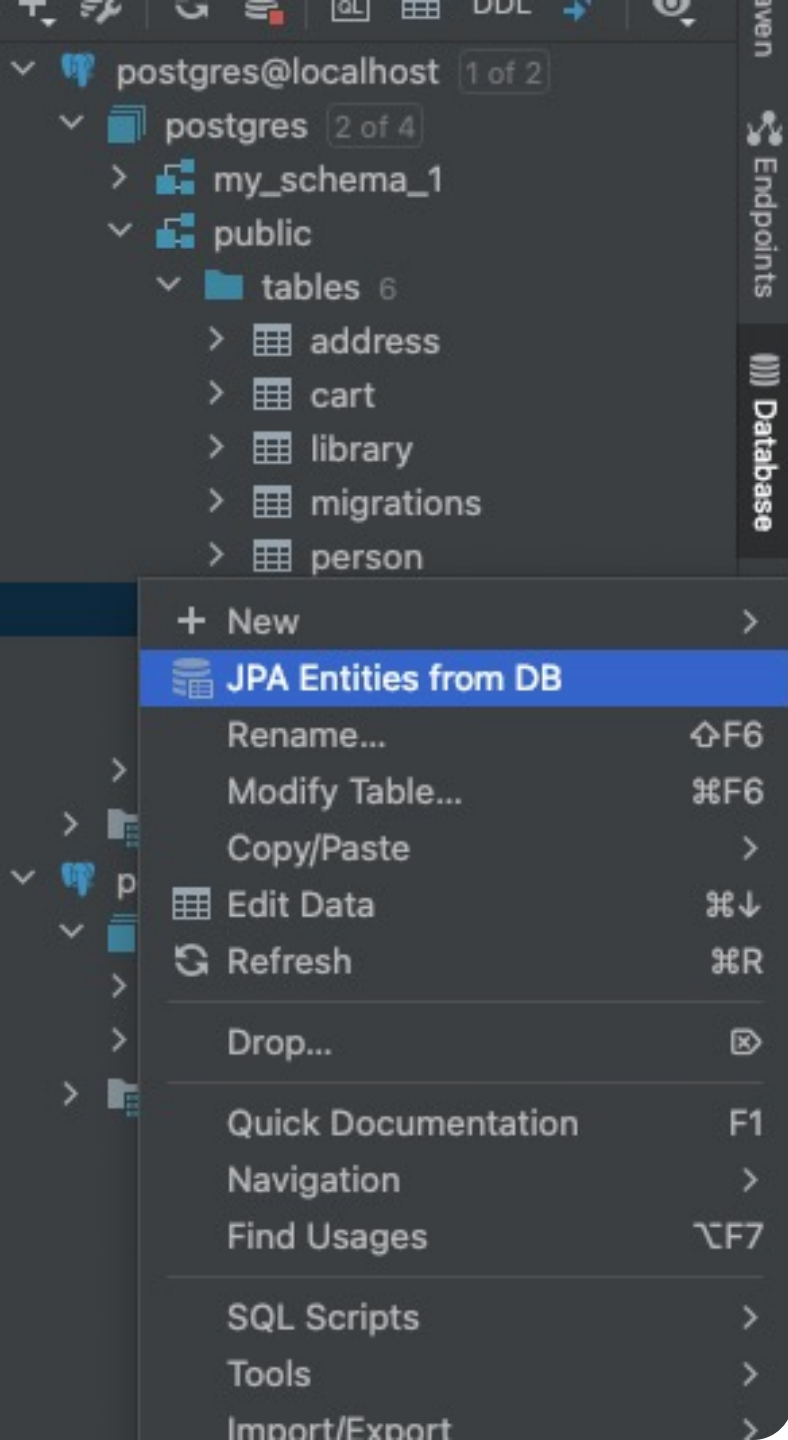
DDL

DDL (Data Definition Language) to język definicji danych – część SQL służąca do **tworzenia, modyfikowania i usuwania struktur bazy danych**, a nie samych danych.

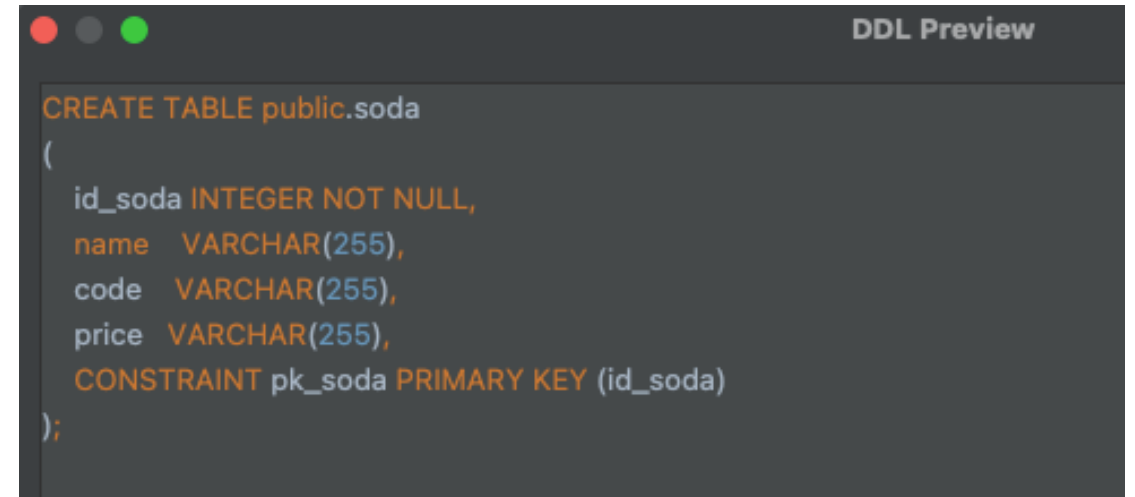
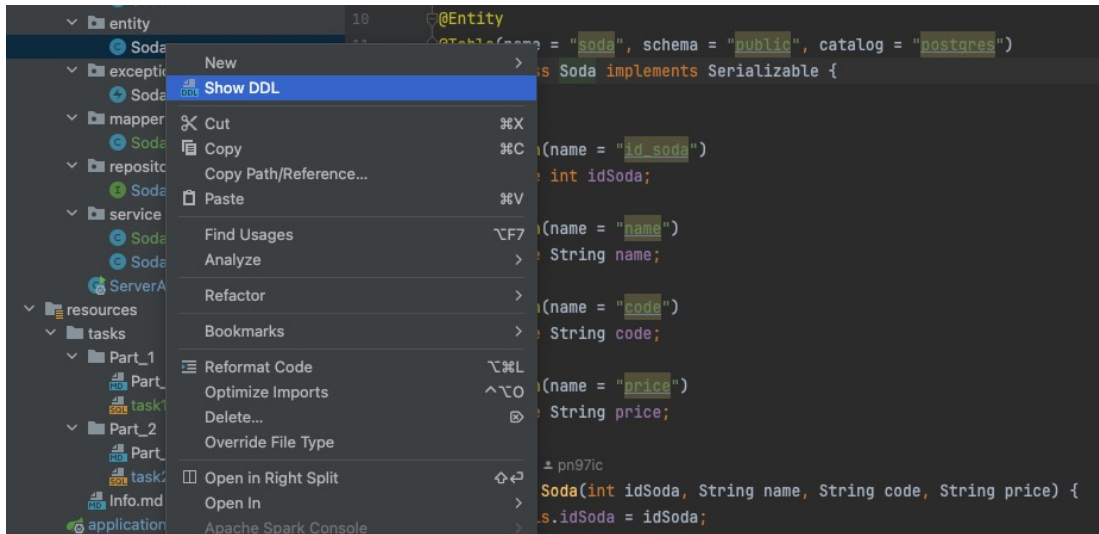
Do czego służy DDL?

- tworzenie nowych tabel i schematów,
- zmiana struktury istniejących tabel,
- usuwanie obiektów bazy,
- przygotowanie struktury bazy dla aplikacji,
- wersjonowanie schematu (np. z Flyway).

Generowanie Encji na podstawie tabeli w bazie



Generowanie DDL na podstawie klasy encji



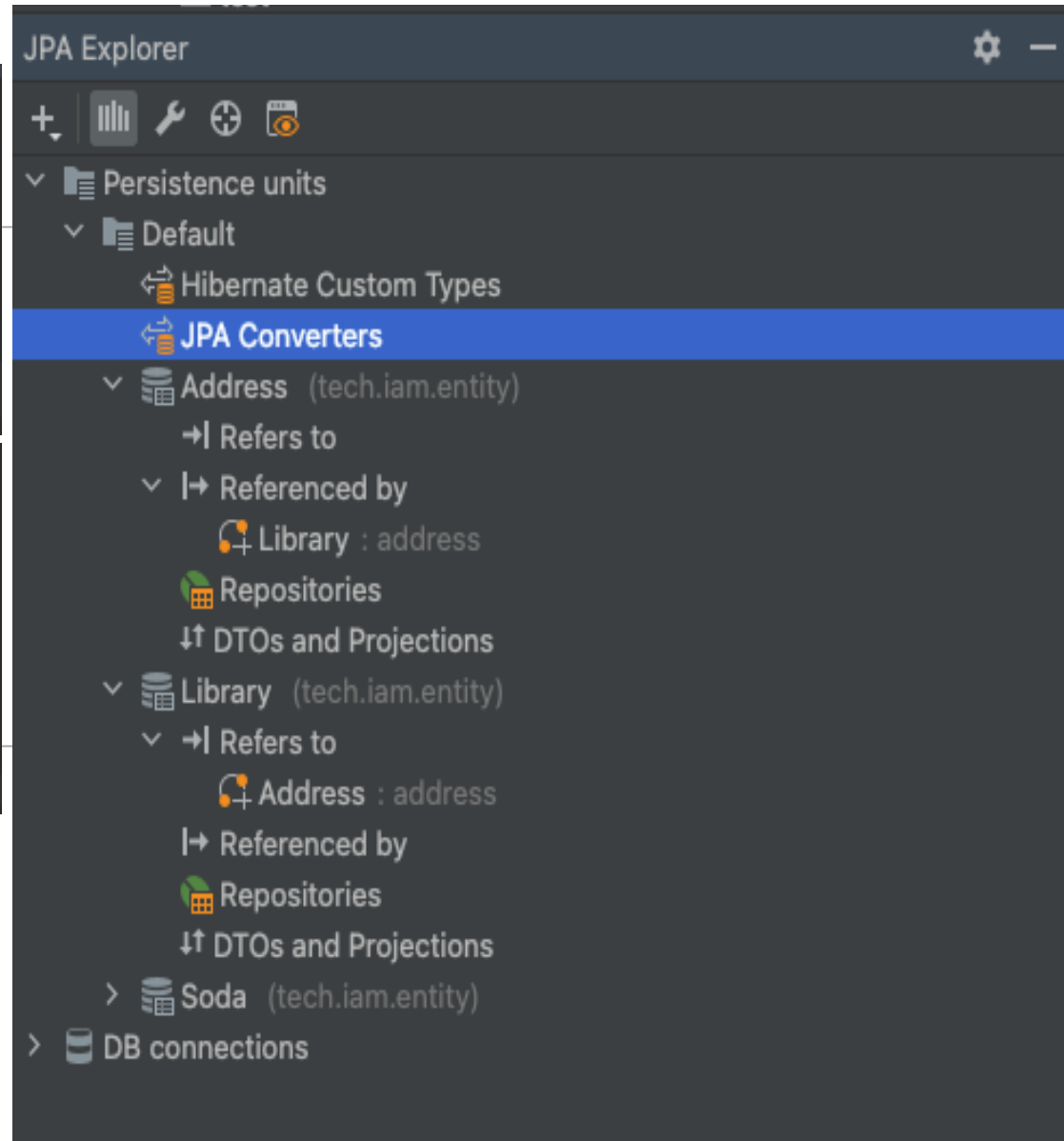
Relacje (Library <-> Address)

```
CREATE TABLE address
(
    id          BIGINT          NOT NULL,
    location    VARCHAR(255)    NOT NULL,
    CONSTRAINT pk_address PRIMARY KEY (id)
);

2
3
4
5
6
7
8
9
10
11
12
13
CREATE TABLE library
(
    id          BIGINT NOT NULL,
    name        VARCHAR(255),
    address_id  BIGINT,
    CONSTRAINT pk_library PRIMARY KEY (id)
);

ALTER TABLE library
ADD CONSTRAINT uc_library_address UNIQUE (address_id);

ALTER TABLE library
ADD CONSTRAINT FK_LIBRARY_ON_ADDRESS FOREIGN KEY (address_id) REFERENCES address (id);
```



Migracja



What? How? Why?

- What?
 - Co rozumiemy pod słowem “migracja/migracje”
 - <https://azure.microsoft.com/pl-pl/resources/cloud-computing-dictionary/what-is-data-migration>
- How?
 - W zautomatyzowany sposób (nigdy “ręcznie”)
- Why?
 - Mieć kontrolę i wiedzieć co się dzieje
 - Wiedzieć, jaką wersję bazy danych mamy na środowisku X
 - Mieć możliwość stworzenia nowego środowiska BD “w mgnieniu oka”

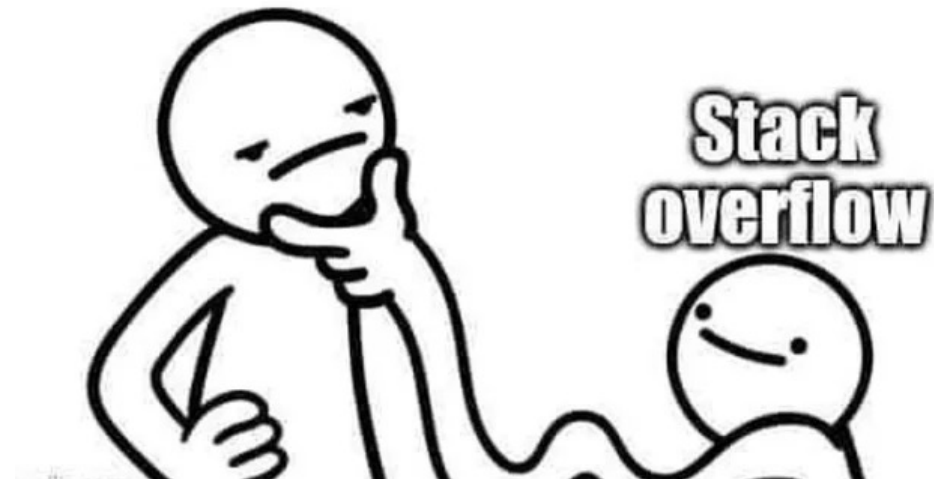
Konfiguracja migratora



- Baza danych Postgresql w Docker:
 - `docker run --name postgres-dev -e POSTGRES_PASSWORD=postgres -p 5400:5432 -d postgres`
- Flyway - Program do migracji¹

1) (Inne (moje ulubione): Liquibase i Flyway – narzędzia stworzone w Java. Wymagają JRE do działania. Nie muszą być używane tylko z Java'owymi projektami)

Summary





Things to avoid



- Czyny karalne
 - **Zapisanie w kodzie konfiguracji połączenia do db wraz z hasłem**
 - `const db = pgp('postgres://postgres:postgres@localhost:5432/postgres')`
 - **Logowanie obiektów zawierających hasło**
 - `console.log("POST /users:", body)`
 - **Insert hasła do bazy danych BEZ HASHOWANIA**
 - `insert into users (first_name, last_name, login, email, password)`
 - **Logowanie odpowiedzi z DB zawierającej hasła** (nawet zahashowane – bo kiedyś w wyniku błędu hashowanie przestanie działać i zalogujesz plaintext (mało prawdopodobne ale jednak))
- W ogóle należy unikać tworzenia mechanizmu zarządzania hasłami na własną rękę! Zbyt wiele rzeczy może pójść nie tak! Temat jest złożony
 - Zakładanie usera / zmiana hasła / polityki haseł / przypomnienie hasła/ wygasanie hasła / zarządzanie rolami (user/admin/auditor/itp...) – na pewno ktoś już to zrobił lepiej.

The background of the slide is a dark blue gradient. It features a complex network of light blue lines connecting various circular nodes. Some nodes are a bright cyan color, while others are a darker blue. The lines and nodes are scattered across the entire frame, creating a sense of a global or digital network.

PODSUMOWANIE

Co warto poznać

- Więcej znajdziecie opisów wraz z przykładami na,
- <https://www.w3schools.com/sql/default.asp>
- Docker <https://bykowski.pl/docker-lista-komand-twoja-podreczna-sciaga/>
- Bazy Danych <https://www.oracle.com/pl/database/what-is-database/#link1>
- <https://www.oracle.com/pl/security/database-security/what-is-data-security/#:~:text=Kontrol%C4%99p%C5%BCytkownik%C3%B3w,celu%C3%B3w%20dotycz%C4%85cych%20zgodno%C5%9Bci.>
- <https://www.plukasiewicz.net/SQL/Syntax>
- <https://www.baeldung.com/sql-injection>
- <https://hackernoon.com/getting-started-debugging-spring-transactions-in-production>

Co warto poznać - Postgres

<https://www.postgresql.org/docs/current/sql-createschema.html>

<https://www.postgresql.org/docs/current/sql-createrole.html>

<https://www.postgresql.org/docs/9.1/sql-grant.html>

<https://tableplus.com/blog/2018/04/postgresql-how-to-grant-access-to-users.html>

<https://www.postgresql.org/docs/current/ddl-schemas.html>

<https://www.postgresql.org/docs/8.0/sql-alteruser.html>

Co warto poznać - EXTRA

- Bonus jak generować gotowy ddl vs encje na podstawie tabeli
- Zarządzanie relacjami w wykorzystaniu hibernate (constraints)



do your thing